MIN SEOHYEON

GAMEPLAY ENGINEER

weare1842@gmail.com
github.com/Seohyeon-Min |
seohyeon-min.github.io/my_portfolio
linkedin.com/in/seohyeon-min

Gameplay engineer who builds gameplay and engine-level systems in C++ and Unity — from object/component architectures and boss pattern systems to collision and rendering pipelines — then debugs and optimizes them under real performance constraints.

"She was able to care just as much about her teammates as she did about her own work." — Jonathan Holmes, DigiPen Instructor

SELECTED EXPERIENCE

TEACHING ASSISTANT - GAME DEVELOPMENT PROJECT I — DIGIPEN KOREA | SPRING 2025

- Supported ~30 students across DigiPen Korea's Game Development Project I cohort with C++ implementation, debugging, and technical problem-solving throughout the term.
- Diagnosed issues across student projects and delivered clear, actionable technical feedback to help teams identify problems and improve their implementations.

MANZO — C++ / OpenGL / GLSL | 2024-2025

- Implemented BPM timing windows, beat/bar counting, and audio-synchronized player movement and boss patterns in a custom C++ engine.
- Built layer-sorted draw queues, framebuffer post-processing for bloom, underwater distortion, god rays, ripples, and transitions, plus particles with linear, curved, radial, spray, random, and player-targeted motion.
- Moved scenario/dialogue ownership into engine-level systems to eliminate dangling-pointer failures; diagnosed and eliminated per-frame redundant collision checks causing severe boss-fight frame drops, restoring stable performance. Largest repository contributor: 366 commits.

NEW MANZO — Unity / C# | 2025-2026

- Designed and implemented a Template Method-based boss pattern architecture (MonsterPatternSO) that fixes shared logic — prepare phase, cooldown, telegraph spawning — in one base class across 21 concrete pattern implementations.
- Built a Composite orchestration layer (CombinePatternSO) that chains sub-patterns into combos at runtime, including combos nested inside combos, through Instantiate-based sequencing with no additional per-combo code.
- Decoupled projectile motion and spawn behavior into standalone interfaces so pattern logic never depends on a concrete projectile implementation; primary C# contributor with 418 of 585 repository commits.

DOUBLE HIT — C++ / raylib | 2024

- Implemented a custom sprite-file parser that loads textures, animation frames, hotspots, and collision shapes, then wires collision components onto GameObjects from parsed data — built within a composition-based GameObject/Component engine.
- Built the texture manager (filename-based caching and dedup, offscreen render-texture mode) and the GameObjectManager driving per-frame update/draw and pairwise collision dispatch across all live objects.
- This implementation became the base Manzo later expanded into layer-based rendering, framebuffer post-processing, CCD, rhythm, and scenario systems.

ADDITIONAL EVIDENCE

BIRD STRIKE - Implemented audio-timeline beat detection, rhythm-synchronized spawning, dynamic attack subdivision, player/crow movement, and atan2 direction logic; also produced original art and audio.
TOO HOT! - Specified GameplayManager and per-stage ScriptableObject data flow, save-range safeguards, chapter selection, and clean-state debug controls within a 130+ item P0-P3 backlog and two-programmer coordination.
STREET TYPER - Specified, evaluated, debugged, and integrated a reusable UI shader workflow for a bilingual typing-combat game published on itch.io and preparing for a Steam release; gameplay code was teammate-authored.

SKILLS

Programming	C++, C#, C, Python, JavaScript; gameplay/engine architecture, design patterns (Template Method, Composite), collision, debugging, memory/lifetime fixes
Systems	Custom C++ engines (raylib, OpenGL), Unity gameplay systems, boss/pattern frameworks, state and data-driven design, performance debugging
Workflow	Git branching and merge review, GitHub Projects/Issues, Notion, CMake, Visual Studio, WSL, profiling, technical specification

EDUCATION

B.S. Computer Science in Real-Time Interactive Simulation | 2026-Present | Expected Graduation: May 2028
Keimyung University / DigiPen Institute of Technology | GPA 3.958/4.0